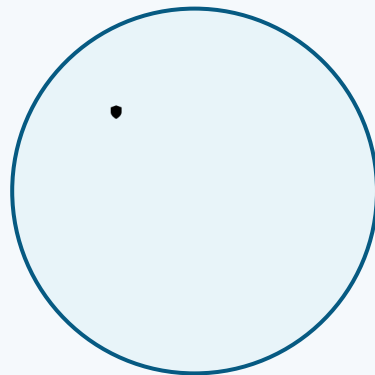


CAWG-TRQP

Verifier Reference Implementation

From content provenance to verifiable trust decisions

v0.17.0



Signatures are necessary. They're not sufficient.



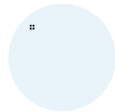
A signed image proves the signature is valid — not that the actor was authorised



Auditors can't replay a verification decision after the fact



Policy lives separately from content: no machine-readable link between them



Multi-registry environments create routing ambiguity and siloed trust

Two standards. One gap. This project bridges it.

C2PA / CAWG

Content Authenticity

- Cryptographic manifests embedded in media files
- Creator assertion binding (who made what, how)
- Tamper-evident provenance chain
- Published by C2PA; creator extensions via CAWG

bridges

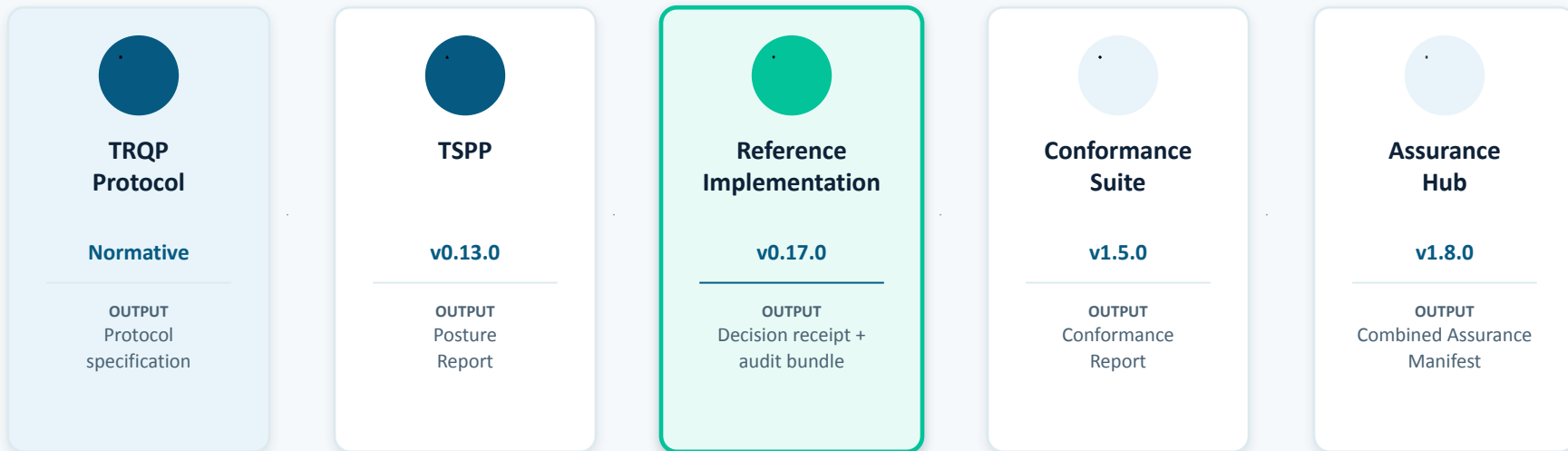
TRQP

Trust Registry Query Protocol

- Machine-readable authorisation and recognition queries
- Decouples policy evaluation from content handling
- Supports offline, cached, and live lookup modes
- Trust registry as governance decision plane

One protocol, five accountable layers.

Authority is distributed by design — no single repository governs the whole decision.



Each layer publishes its own evidence; cross-stack use pins versions or commit SHAs into the evidence bundle.

WHAT THIS PROJECT IS

A reference implementation — not a product, a proof of architecture.

This repository shows how TRQP can operate as the governance decision plane in a CAWG/C2PA verification workflow. Every decision is captured as a receipt — a working, testable model of how trust infrastructure should behave.



Verifiable

Decisions are captured as a receipt and can be exported, validated, and independently replayed.



Portable

Audit bundles carry all evidence needed to re-examine a decision, including descriptor and freshness evidence.



Composable

Policy, content, and process verification are separated by design.

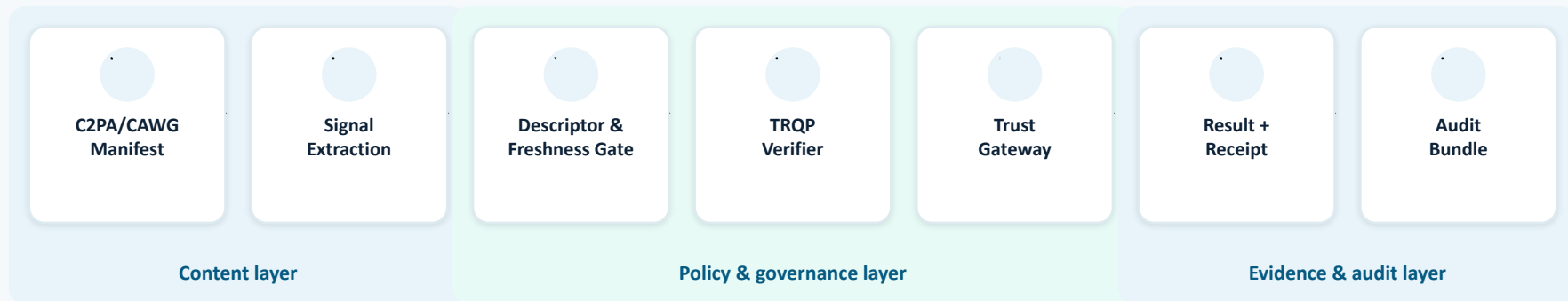


Deployable

Three profiles — edge, standard, high-assurance — plus an HTTP service with Docker packaging.

HOW IT WORKS

From manifest to verified decision: the verification pipeline.



Process appraisal is optional and layered in at the Verifier stage — the gateway routes to policy services per `authority_id`, enabling multi-registry and federated deployments without re-architecting each verifier.

One verifier. Three assurance postures.



EDGE

Offline / Constrained

- Signed snapshot — no live network call
- Handheld, IoT, disconnected environments
- Fails on snapshot descriptor policy when configured



STANDARD

Platform / Service

- Cache-first, live lookup on miss
- Typical enterprise platform deployment
- Observes descriptor evidence



HIGH ASSURANCE

Regulated / Audit-Sensitive

- Live policy lookup — always
- Full audit bundle mandatory
- Fail-closed on policy, revocation & gateway-route descriptor defects

Evidence isn't a log. It's a machine-verifiable artifact.

Every verification issues a decision receipt. On request, it exports as a deterministic, schema-validated audit bundle — portable and replay-ready across implementations.

What was evaluated?

request_summary + replay_inputs.request

What evidence supported it?

policy_evidence + descriptor & freshness evidence + policy epoch timestamp

What decision was reached?

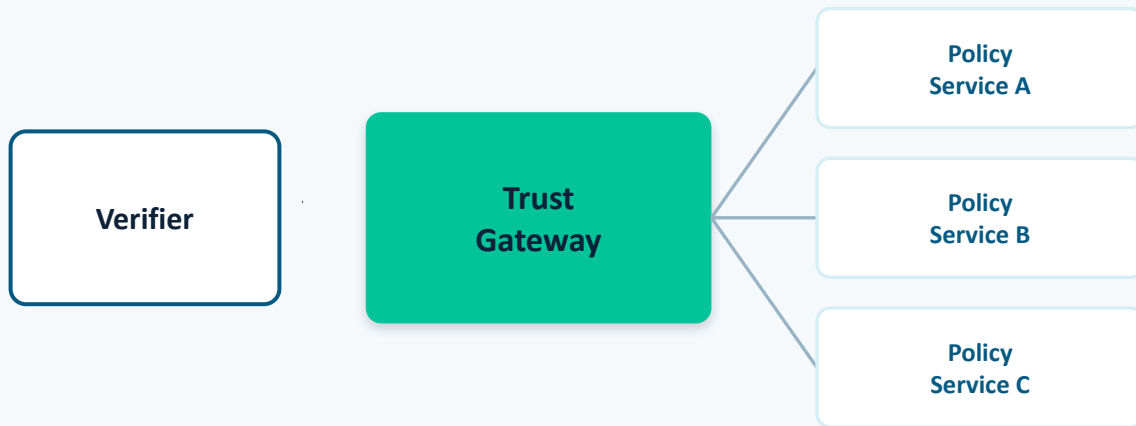
verification_result with full outcome, on a signed decision receipt

Can it be replayed?

Schema validation + digest check + canonical fixture replay tooling

Policy routing as a first-class concern.

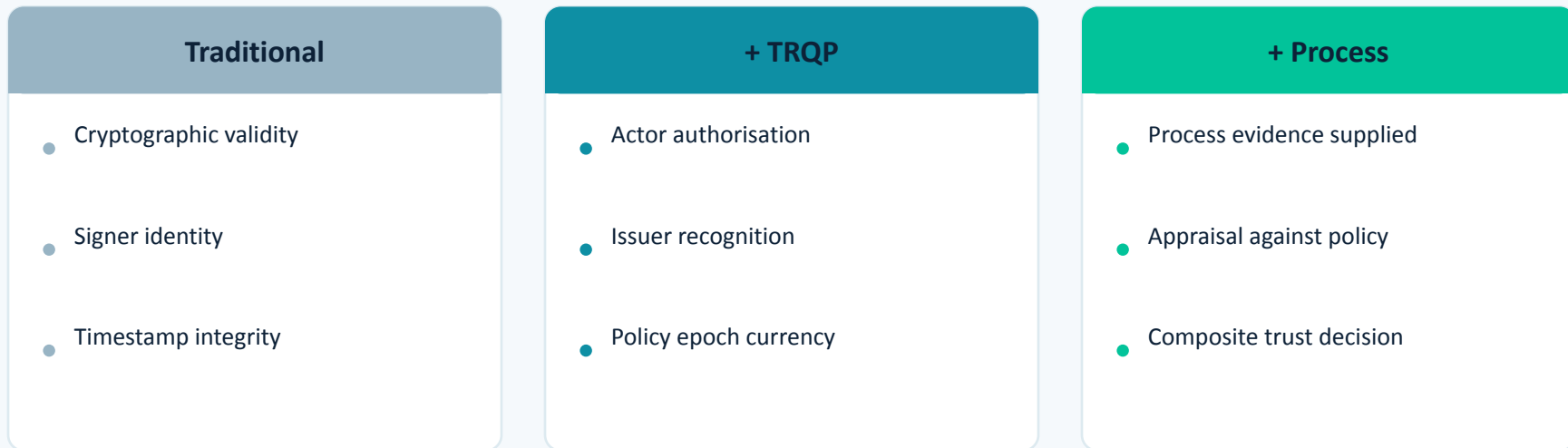
The trust gateway sits between the verifier and one or more policy services. It routes by `authority_id`, mediates across policy domains, and produces a machine-readable mediation trace for every lookup — now first-class evidence, not infrastructure noise.



Routes by `authority_id` · Produces mediation trace · Enables federated topologies

Beyond identity: did the process meet the standard?

Most verification stops at 'is this signature valid?' This project adds a third dimension: 'did the process behind this action meet the expected governance standard?'



Standards need working code to become real infrastructure.



Implementer clarity

Spec text alone leaves too much ambiguous. Running code settles design questions that prose cannot.



Interoperability proofs

Canonical fixture packages and a compatibility matrix give implementers concrete test targets, not just aspirations.



Upgrade path, not mandate

Process evidence and descriptor policy are optional and configurable. Organisations adopt incrementally.



Separation of concerns

Policy, content, and process are decoupled. Each layer — and each repository in the stack — can evolve without breaking the others.

Who should be paying attention to this work?



Media & Publishing Platforms

Provenance and creator authorisation at scale; audit-grade evidence retention for regulatory exposure.



Enterprise IT / Architects

Policy mediation patterns that avoid hard-coding trust into every service; deployment profiles for constrained environments.



Standards Bodies & Spec Editors

Working code that validates spec assumptions and surfaces ambiguity before it becomes interoperability debt.



Regulated Industries

Process-aware verification that connects content claims to auditable governance evidence — without collapsing all postures into one.

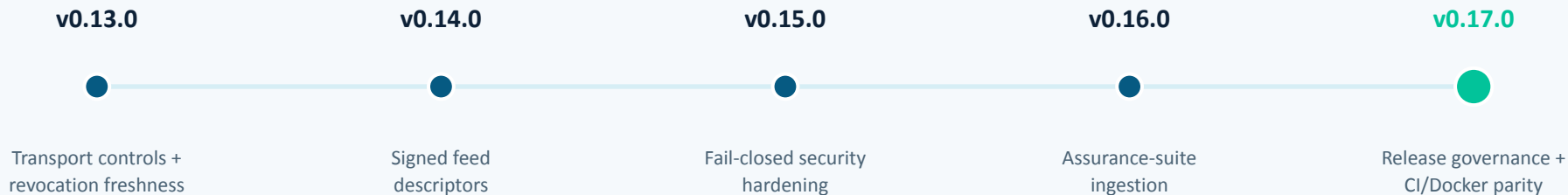


Government / Policy Teams

Separation of policy from execution enables jurisdiction-specific trust registries within a common protocol.

ROADMAP

Where the implementation has been — and where it's heading.



NEXT HORIZON — FUTURE WORK



The implementation is open. The design questions are live.

1

Run the implementation

`pip install -r requirements-lock.txt && pip install -e .` — three profiles, immediate CLI output. Or docker compose up --build for the HTTP service.

2

Explore decision receipts & audit bundles

Export a bundle, validate its schema, and replay it. Understand what deterministic, cross-implementation assurance means in practice.

3

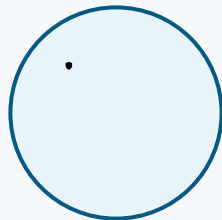
Engage on open issues

A real binary C2PA parser backend, an expanded negative-vector library, and cross-repository alignment across the TRQP stack — concrete problems to work on.

Standards context

- C2PA · Content Authenticity
- CAWG · Creator Assertions
- TRQP · Trust Registry Query Protocol
- W3C DIDs & Verifiable Credentials
- ISO/IEC 18013-5 (mDL reference point)

Full docs: sankarshanmukhopadhyay.github.io/cawg-trqp-verifier-refimpl



The question isn't whether trust infrastructure is needed.

It's whether we build it deliberately — or inherit it by accident.

CAWG-TRQP Reference Implementation

v0.17.0 · Open

github.com/cawg · thetrustgraph.substack.com
sankarshanmukhopadhyay.github.io/cawg-trqp-verifier-refimpl